SOR4008 Bayesian simulation study

Andrew Jones

November 8, 2026

1 Introduction

Linear regression is one of the most widely used statistical models for relating a response variable to one or more covariates. In a Bayesian framework the unknown regression coefficients and the error variance are treated as random variables, and prior information about these parameters is combined with the data to produce a posterior distribution. This posterior distribution can then be used to obtain point estimates, uncertainty intervals and predictions, and to compare with maximum likelihood estimates [2, 3].

Even though the model considered here is conjugate and the posterior can be derived analytically, it is still useful to use simulation-based methods such as Markov chain Monte Carlo (MCMC), because these are the tools we rely on in more complex models where analytic solutions are not available. MCMC algorithms generate approximate samples from the posterior distribution and are now standard tools for Bayesian computation. Different algorithms can behave quite differently in practice, so it is important to understand their accuracy, mixing behaviour and computational cost, even for relatively simple models.

In this project a Bayesian linear regression model with two covariates is studied using simulated data with known true parameter values. Independent normal priors are placed on the regression coefficients α, β_1, β_2 and a Gamma prior on the error precision τ, which corresponds to the standard conjugate normal–Gamma model for Bayesian linear regression. The methods used to estimate the parameters are maximum likelihood, a custom Gibbs sampler, a custom Metropolis–Hastings (MH) algorithm, a custom importance sampling (IS) algorithm, and the `MCMCregress` function from the `MCMCpack` package in R. Using simulated data allows the performance of these methods to be assessed in terms of how well they approximate the true parameter values under repeated sampling, as demonstrated in Section 3.1 for the Gibbs sampler.

The main aims are to compare the posterior estimates from these methods, to study the mixing behaviour of the MCMC algorithms using traceplots and autocorrelation functions, and to compare their computational efficiency. The Gibbs sampler is of particular interest here because the model is conjugate, so all the full conditional distributions for each parameter are available in closed form, while the MH and IS approaches illustrate some of the practical challenges that arise when the algorithm or proposal distribution is not well matched to the posterior.

The rest of the report is organised as follows. Section 2 describes the model, priors and computational methods in more detail, Section 3 presents the results of the simulations and method comparisons, and Section 4 concludes with a discussion of the results. Note:

All custom methods are coded in R and can be accessed through the Github link in the Bibliography.

2 Methods

2.1 Model and priors

We consider the simple linear regression model

$$y_i = \alpha + \beta_1 x_{1i} + \beta_2 x_{2i} + \varepsilon_i, \qquad \varepsilon_i \sim \mathcal{N}(0, 1/\tau). \tag{1}$$

where α, β_1, β_2 are regression parameters and τ is the precision.

For the main experiments a dataset of size $n = 100$ was simulated. The covariates were generated independently from a uniform distribution on $[0, 10]$ and rounded to 2 decimal places,

$$x_{1i} \sim \text{Unif}(0, 10), \qquad x_{2i} \sim \text{Unif}(0, 10), \qquad i = 1, \ldots, n.$$

The responses were then generated using fixed true parameter values $\alpha_{\text{true}} = 1.5$, $\beta_{1_{\text{true}}} = 2$, $\beta_{2_{\text{true}}} = 5$, $\tau_{\text{true}\,=\,1}$.

A standard Bayesian linear regression set–up was used with independent normal priors for the regression parameters and a Gamma prior for the precision,

$$\alpha \sim \mathcal{N}(m_\alpha, s_\alpha^2), \qquad \beta_1 \sim \mathcal{N}(m_{\beta_1}, s_{\beta_1}^2), \qquad \beta_2 \sim \mathcal{N}(m_{\beta_2}, s_{\beta_2}^2),$$
$$\tau \sim \text{Gamma}(a_\tau, b_\tau).$$

In the main analysis, weakly informative priors were chosen centred near the true values so that the posterior is driven mainly by the data. The prior parameters were set to:

$$m_\alpha = 0, \qquad m_{\beta_1} = 0, \qquad m_{\beta_2} = 2,$$
$$s_\alpha^2 = s_{\beta_1}^2 = s_{\beta_2}^2 = 100,$$
$$a_\tau = b_\tau = 0.5.$$

The same prior specification is used in all Bayesian methods (Gibbs sampler, Metropolis–Hastings and importance sampling), and for the MCMCpack implementation I matched the prior as closely as possible.

2.2 Maximum Likelihood

As a baseline I also computed the maximum likelihood (ML) estimates for the model, which in this example reduces to ordinary least squares. In R I fitted Equation 1 using the $lm(y \sim x_1 + x_2)$ function. The ML estimates of α, β_1, and β_2 are taken directly from the fitted coefficients, and the error variance is estimated from the residual sum of squares. The precision estimate is then $\hat{\tau} = \frac{1}{\hat{\sigma}^2}$. These ML estimates are compared with the Bayesian posterior means in the results.

2.3 Importance sampling

Importance sampling (IS) was implemented to approximate the joint posterior $(\alpha, \beta_1, \beta_2, \tau)$. In this project I used the prior as the proposal distribution. Let $\theta = (\alpha, \beta_1, \beta_2, \tau)$. The goal is to approximate the posterior $p(\theta \mid y)$ by a weighted sample.

To generate M approximate samples from the posterior, I used the following steps:

1. Draw K candidate parameter values from the prior (with $K \gg M$):

$$\theta^{(k)} = \big(\alpha^{(k)}, \beta_1^{(k)}, \beta_2^{(k)}, \tau^{(k)}\big) \sim p(\theta), \qquad k = 1, \ldots, K,$$

   where the components are sampled independently from their priors,

$$\alpha^{(k)} \sim \mathcal{N}(m_\alpha, s_\alpha^2), \quad \beta_1^{(k)} \sim \mathcal{N}(m_{\beta_1}, s_{\beta_1}^2), \quad \beta_2^{(k)} \sim \mathcal{N}(m_{\beta_2}, s_{\beta_2}^2), \quad \tau^{(k)} \sim \text{Gamma}(a_\tau, b_\tau).$$

2. For each k, compute an unnormalised importance weight proportional to the likelihood:

$$\tilde{w}_k \propto p\big(y \mid \theta^{(k)}\big) = \prod_{i=1}^{n} \left(\frac{\tau^{(k)}}{2\pi}\right)^{1/2} \exp\left\{-\frac{\tau^{(k)}}{2}\big(y_i - \alpha^{(k)} - \beta_1^{(k)} x_{1i} - \beta_2^{(k)} x_{2i}\big)^2\right\}.$$

   In practice, I used the equivalent expression

$$\tilde{w}_k = \left(\frac{\tau^{(k)}}{2\pi}\right)^{n/2} \exp\left\{-\frac{\tau^{(k)}}{2} \sum_{i=1}^{n} \big(y_i - \alpha^{(k)} - \beta_1^{(k)} x_{1i} - \beta_2^{(k)} x_{2i}\big)^2\right\}.$$

3. Normalise the weights so that they sum to one:

$$w_k = \frac{\tilde{w}_k}{\sum_{j=1}^{K} \tilde{w}_j}, \qquad k = 1, \ldots, K.$$

4. Finally, obtain M approximate posterior draws by resampling from the $\theta^{(k)}$ with probabilities w_k:

$$\theta^{*(1)}, \ldots, \theta^{*(M)} \sim \{\theta^{(1)}, \ldots, \theta^{(K)}\} \quad \text{with } P(\theta^{*(m)} = \theta^{(k)}) = w_k.$$

The resampled values $\theta^{*(m)}$ are then treated as an approximate sample from the posterior and used to estimate posterior means and credible intervals, and to compare accuracy and runtime with the other methods [4].

2.4 Metropolis–Hastings algorithm

A Metropolis–Hastings (MH) algorithm is used to sample from the joint posterior distribution of $\theta = (\alpha, \beta_1, \beta_2, \tau)$. The target density is the posterior

$$\pi(\theta) = p(\theta \mid y) \propto p(y \mid \theta)\, p(\theta),$$

where $p(y \mid \theta)$ is the Gaussian likelihood from the linear model and $p(\theta)$ is the prior described in Section 2.2.

The MH algorithm constructs a Markov chain $\{\theta^{(t)}\}_{t\geq 0}$ with stationary distribution $\pi(\theta)$ by repeatedly proposing new values from a chosen proposal distribution and accepting or rejecting them with a suitable probability. Let $q(\theta^* \mid \theta)$ denote a proposal density for a new state θ^* given the current state θ.

The MH algorithm can be summarised as follows.

1. **Initialisation.** Choose a starting point
$$\theta^{(0)} = (\alpha^{(0)}, \beta_1^{(0)}, \beta_2^{(0)}, \tau^{(0)}),$$
for example by drawing from the prior or setting reasonable initial values.

2. **For** $t = 0, 1, \ldots, T-1$:

   (a) **Propose a new value.** Draw a candidate $\theta^* = (\alpha^*, \beta_1^*, \beta_2^*, \tau^*)$ from the proposal distribution
   $$\theta^* \sim q(\,\cdot \mid \theta^{(t)}).$$

   (b) **Compute the acceptance probability.** The Metropolis–Hastings acceptance ratio is
   $$r = \frac{\pi(\theta^*)\, q(\theta^{(t)} \mid \theta^*)}{\pi(\theta^{(t)})\, q(\theta^* \mid \theta^{(t)})} = \frac{p(y \mid \theta^*)\, p(\theta^*)\, q(\theta^{(t)} \mid \theta^*)}{p(y \mid \theta^{(t)})\, p(\theta^{(t)})\, q(\theta^* \mid \theta^{(t)})},$$
   and the acceptance probability is
   $$\alpha(\theta^{(t)}, \theta^*) = \min\{1,\, r\}.$$

   (c) **Accept or reject.** Draw $u \sim \mathrm{Unif}(0, 1)$. If $u \leq \alpha(\theta^{(t)}, \theta^*)$ then accept the proposal and set $\theta^{(t+1)} = \theta^*$; otherwise reject the proposal and keep the current state, $\theta^{(t+1)} = \theta^{(t)}$.

This procedure defines a Markov chain whose stationary distribution is the posterior $\pi(\theta)$ [4]. For its implementation in this project, the proposal distribution $q(\theta^* \mid \theta^{(t)})$ is chosen to be the prior. The sampler is run for $50,000$ iterations with a burn–in of $10,000$ iterations, and the post–burn–in draws are used to estimate posterior distributions for the parameters and to produce the trace plots and autocorrelation functions in the results section.

2.5 Gibbs sampler

A Gibbs sampler is used as the main Bayesian method to sample from the joint posterior distribution of $\theta = (\alpha, \beta_1, \beta_2, \tau)$. As before, the target density is

$$\pi(\theta) = p(\theta \mid y) \propto p(y \mid \theta)\, p(\theta),$$

where $p(y \mid \theta)$ is the Gaussian likelihood from the linear model and $p(\theta)$ is the prior specified in Section 2.2.

Under the chosen conjugate priors, the full conditional distributions of the parameters have standard forms: the conditionals for α, β_1 and β_2 are normal, and the conditional for τ is Gamma. The Gibbs sampler exploits this by updating each parameter from its

full conditional distribution, given the current values of the others. This avoids the need for accept–reject steps and ensures that every proposed value is accepted.

Let $\theta^{(t)} = (\alpha^{(t)}, \beta_1^{(t)}, \beta_2^{(t)}, \tau^{(t)})$ denote the state of the chain at iteration t, and T the number of iterations. The Gibbs sampler algorithm can be summarised as:

1. **Initialisation.** Choose a starting point
$$\theta^{(0)} = (\alpha^{(0)}, \beta_1^{(0)}, \beta_2^{(0)}, \tau^{(0)}),$$
for example by drawing from the prior or using simple initial guesses.

2. **For** $t = 0, 1, \ldots, T-1$:

   (a) **Update** α**.** Draw
   $$\alpha^{(t+1)} \sim p\big(\alpha \mid \beta_1^{(t)}, \beta_2^{(t)}, \tau^{(t)}, y\big),$$
   from its normal full conditional.

   (b) **Update** β_1**.** Draw
   $$\beta_1^{(t+1)} \sim p\big(\beta_1 \mid \alpha^{(t+1)}, \beta_2^{(t)}, \tau^{(t)}, y\big),$$
   from its normal full conditional.

   (c) **Update** β_2**.** Draw
   $$\beta_2^{(t+1)} \sim p\big(\beta_2 \mid \alpha^{(t+1)}, \beta_1^{(t+1)}, \tau^{(t)}, y\big),$$
   from its normal full conditional.

   (d) **Update** τ**.** Draw
   $$\tau^{(t+1)} \sim p\big(\tau \mid \alpha^{(t+1)}, \beta_1^{(t+1)}, \beta_2^{(t+1)}, y\big),$$
   from its Gamma full conditional.

The means and variances for the normal conditionals and the shape and rate for the Gamma conditional are computed in the R code at each iteration, and then used to draw the new values. In the main analysis the Gibbs sampler is run for $T = 50{,}000$ iterations with a burn-in period of 10,000 iterations, and the remaining post-burn-in draws are used as approximate samples from the posterior.

Convergence is assessed using traceplots, autocorrelation functions and running means of the post-burn-in samples in the results section. To study the behaviour of the Gibbs estimator under repeated sampling, the whole data generation and Gibbs fitting procedure is also repeated for 1,000 simulated datasets, and the posterior means and credible intervals from each run are summarised across simulations.

2.6 Package Implementation

To check the custom Gibbs implementation and compare with an R package, the model was also fitted using the *MCMCregress* function from the *MCMCpack* package in R. The same data, model formula ($y \sim x_1 + x_2$), priors, number of iterations and burn-in were used, and the posterior output from MCMCregress was compared with the custom Gibbs sampler in the results.

3 Results

3.1 Gibbs Sampler results

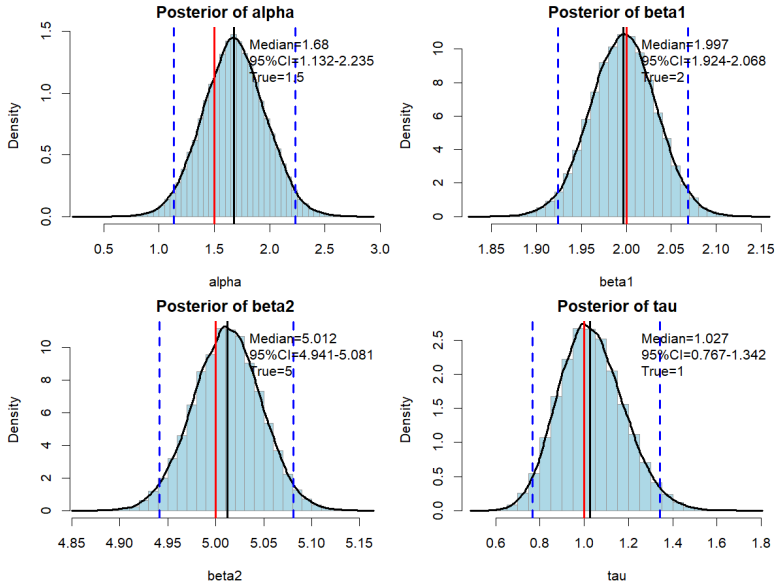


Figure 1: Posterior distributions for α, β_1, β_2, and τ obtained from the Gibbs sampler after $50,000$ iterations and a burn-in of $10,000$. Histograms show sampled posterior densities; dashed vertical lines indicate 95% credible intervals, and solid red lines mark the true parameter values used to generate the data.

Figure 1 shows that for a simulated dataset the Gibbs sampler produces posterior distributions that are well behaved, approximately normal, and centred very close to the true value for each parameter. Table 1 shows that the posterior estimate for each parameter is very close to the true values $\alpha = 1.5$, $\beta_1 = 2$, $\beta_2 = 5$, and $\tau = 1$, and that the true values lie comfortably within the 95% credible intervals.

To assess the bias and variability of the Gibbs estimates under repeated sampling, the data generation and fitting procedure was repeated for 1000 simulated datasets, keeping the covariates x_1 and x_2 fixed and regenerating only the responses y.

Figure 2 shows the resulting sampling distributions of the posterior means across these 1000 simulations, together with the true values and the average posterior mean. For all four parameters the sampling distribution is tightly concentrated around the true value. The average posterior mean over all simulations lies essentially on the true parameter value, suggesting negligible bias of the Gibbs estimator in this example.

Convergence diagnostics for the single simulated dataset support the visual information from Figure 1. The post–burn–in trace plots in Figure 3 show rapid movement around a stable region with no visible drifts or trends for any parameter, and good mixing around the true values, suggesting convergence. The autocorrelation functions in Figure 4 decay to zero quickly for all parameters, indicating low serial dependence between successive Gibbs draws.

Figure 5 shows that the running means of the post–burn-in Gibbs samples stabilise quickly

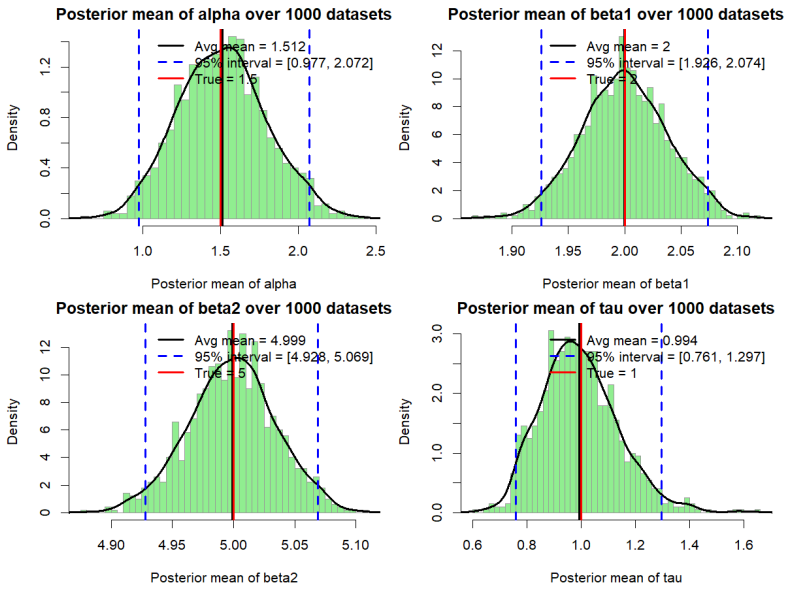


Figure 2: Distribution of α, β_1, β_2, τ across 1000 simulated datasets with 95% credible intervals. Each histogram shows the sampling distribution of the posterior mean under repeated data generation of the responses, while the inputs are kept the same. The red vertical line marks the true parameter value, and the black line shows the average posterior mean across simulations.

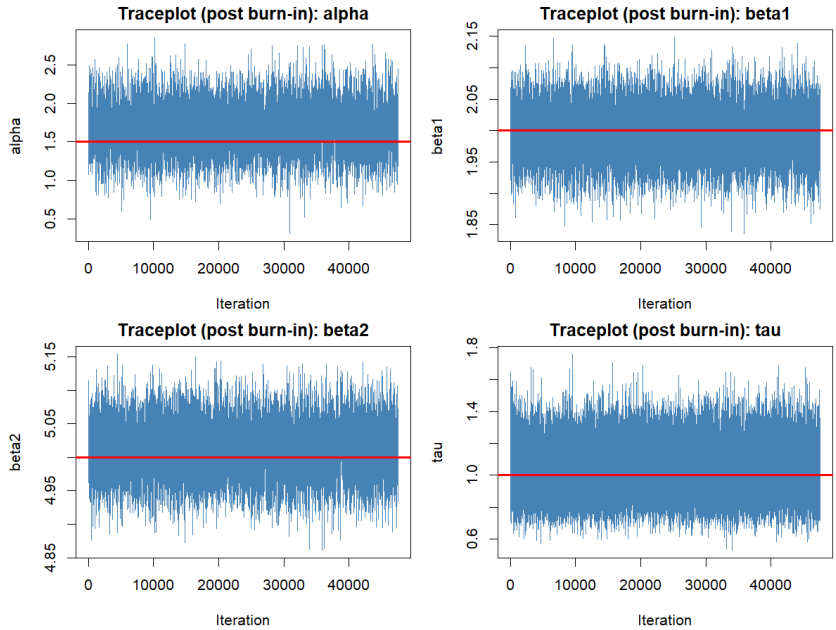


Figure 3: Post–burn-in traceplots for α, β_1, β_2, and τ from the Gibbs sampler. Blue lines show the Gibbs sampler draws, and red horizontal lines indicate the true parameter values. The chains mix well around the correct values with no visible trends, suggesting good convergence.

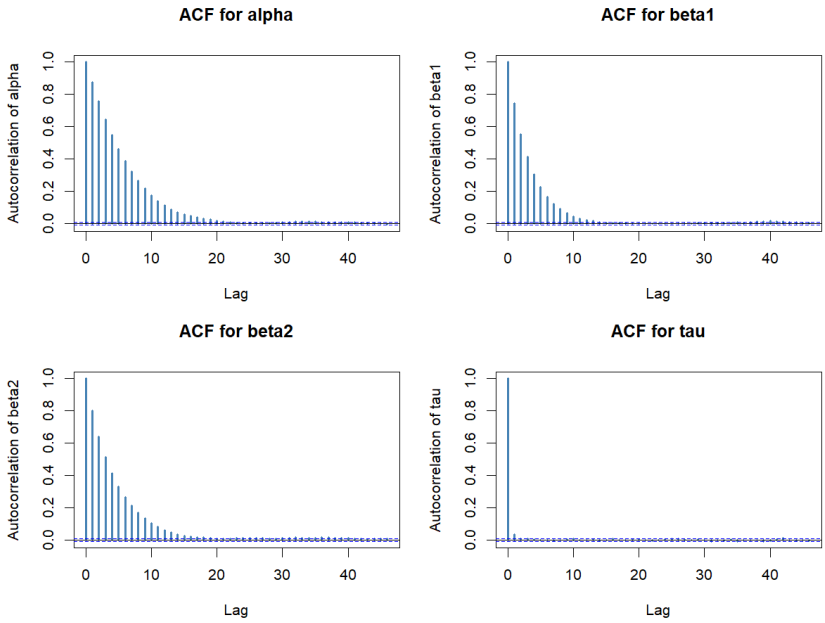


Figure 4: Autocorrelation functions (ACFs) for the post–burn-in samples of α, β_1, β_2, and τ. Low autocorrelation across lags indicates good mixing of the Gibbs sampler.

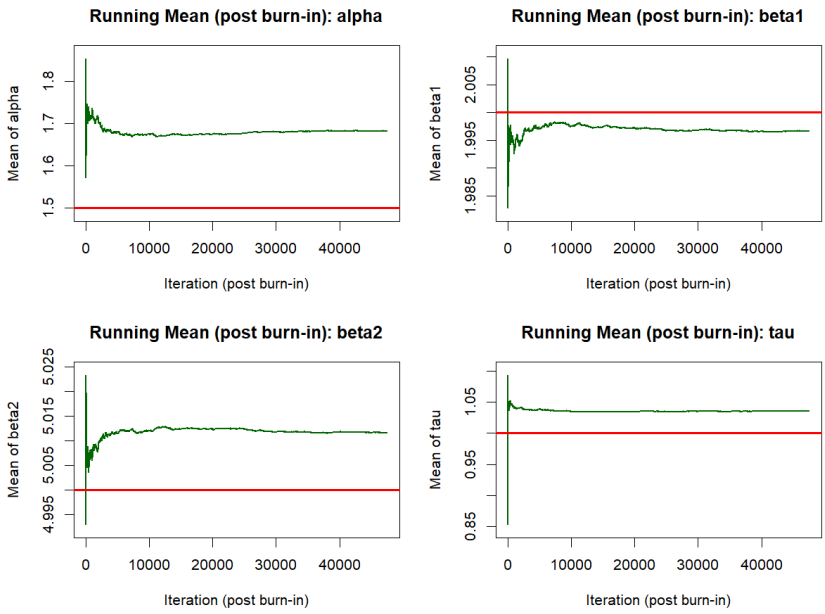


Figure 5: Running means of the post–burn-in Gibbs samples for α, β_1, β_2, and τ. Each curve stabilises, indicating convergence of the Monte Carlo estimate of the posterior mean. The red horizontal lines shows the true parameter values, where slight differences between the converged running mean and the true parameter value is due to sampling noise in the simulated dataset.

around a constant value, which is slightly different from the true value due to noise in the simulated dataset.

	alpha	beta1	beta2	tau
mean	1.6845	1.9963	5.0115	1.0348
sd	0.2832	0.0367	0.0358	0.1475
q025	1.1323	1.9236	4.9406	0.7672
q975	2.2365	2.0681	5.0808	1.3429

Table 1: Posterior summary: custom Gibbs sampler

	alpha	beta1	beta2	tau
mean	1.6789	1.9972	5.0117	1.0369
sd	0.2814	0.0365	0.0356	0.1482
q025	1.1252	1.9266	4.9421	0.7676
q975	2.2293	2.0691	5.0816	1.3477

Table 2: Posterior Summary: Package Gibbs Sampler

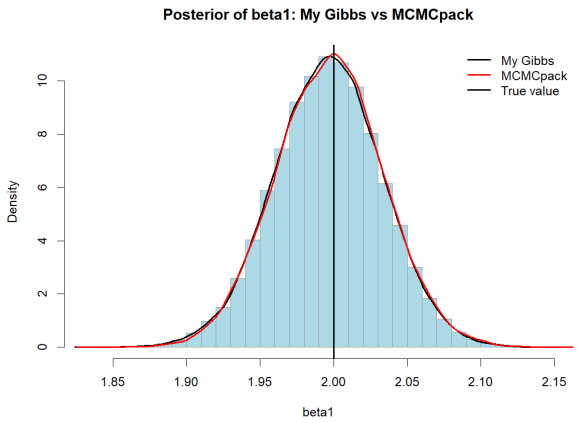


Figure 6: Posterior distribution of β_1 from the custom Gibbs sampler (black) and from MCMCregress in MCMCpack (red), for a single simulated dataset. The two curves overlap almost perfectly, demonstrating the only real difference between methods is the run time, with elapsed time for my Gibbs sampler giving $t_{\text{gibbs}} = 2.20s$, and the package elapsed time giving $t_{\text{package}} = 0.36s$. The vertical line shows the true β_1 value used to generate the data.

Finally, I compared the custom Gibbs implementation with the MCMCregress function from the MCMCpack package in R, using the same number of iterations and the same burn-in on the same dataset. Tables 1–3 and Figure 6 show that the two methods produce almost identical posterior summaries. For example, the posterior means from the custom Gibbs sampler and MCMCpack for β_1 are 1.9963 and 1.9972 respectively, and the overlaid posterior distributions in Figure 6 are essentially indistinguishable.

The main difference between the two implementations is the computational time, shown in Table 3. My R implementation required 2.20 seconds to run 10,000 iterations, whereas MCMCpack completed the same number of iterations in 0.36 seconds.

3.2 Importance sampler results

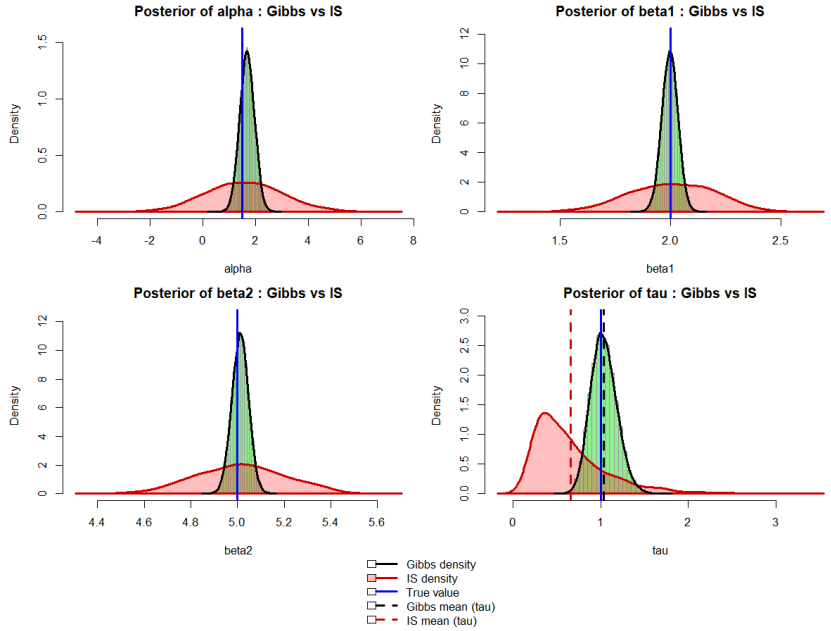


Figure 7: Posterior distributions of α, β_1, β_2, and τ from the custom Gibbs sampler (black density, green histogram) and Importance sampling (red shaded density) for a single simulated dataset. Vertical blue lines show the true parameter values. The two samplers give almost identical parameter estimates, apart from for τ, highlighting the limitations of Importance sampling for this example

Figure 7 compares the posterior distributions obtained from the Gibbs sampler and the importance sampler for a single simulated dataset. Both methods give posterior estimates that are centred close to the true values, so the posterior means are very similar. However, the IS approximation is much flatter and more spread out than the Gibbs posterior, which indicates that, with the chosen proposal and number of samples, importance sampling produces a less concentrated posterior and therefore less precise inference for the parameters.

For τ the performance of IS is noticeably worse. The posterior density for τ in Figure 7 is both flatter and shifted to the left away from the Gibbs posterior and the true value $\tau = 1$. This is consistent with the numerical results in Table 3, where the estimate of τ is the furthest from the true value and the overall root mean squared error (RMSE) for IS is the largest among all methods. In addition, IS is by far the slowest method with a runtime of about 936 seconds.

In this example I used the prior distribution as the proposal distribution. For this four parameter model this works quite poorly as most proposals have very small likelihood, which leads to highly variable weights and a very small effective sample size, so a very large number of proposals is required to get a reasonable approximation. Overall, IS gives reasonable posteriors for α, β_1 and β_2, but it is much less precise than Gibbs, performs poorly for τ, and is also computationally inefficient.

	alpha	beta1	beta2	tau	Time_sec	RMSE
True values	1.5000	2.0000	5.0000	1.0000	NA	NA
Maximum likelihood	1.6771	1.9974	5.0118	1.0709	0.09	0.0956
Gibbs (mine)	1.6818	1.9967	5.0116	1.0347	2.20	0.0928
Gibbs (MCMCpack)	1.6789	1.9972	5.0117	1.0369	0.36	0.0915
Metropolis–Hastings	1.6734	1.9979	5.0121	1.0779	2.36	0.0953
Importance sampling	1.5893	2.0087	5.0168	0.6527	936.00	0.1795

Table 3: True parameter values, point/posterior estimates, runtimes (seconds), and overall RMSE for each method.

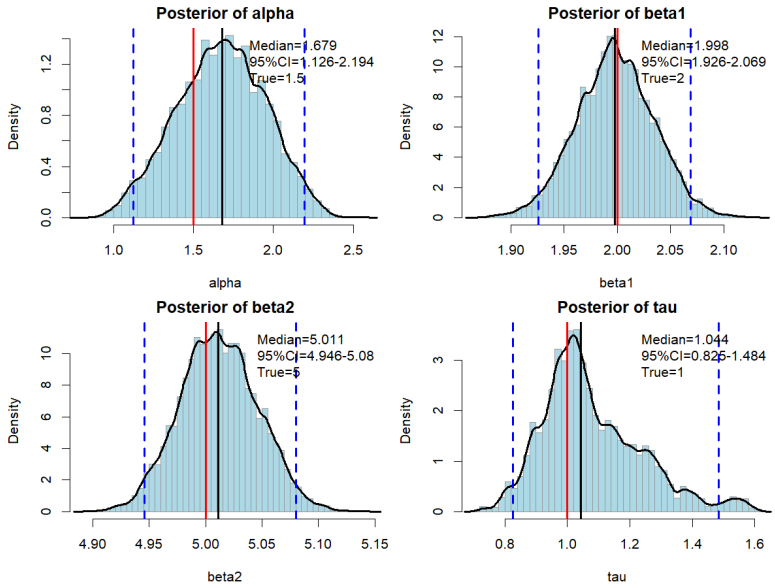


Figure 8: Posterior distributions for α, β_1, β_2, and τ obtained from the Metropolis-Hastings algorithm after $50,000$ iterations and a burn-in of $10,000$. Histograms show sampled posterior densities; dashed vertical lines indicate 95% credible intervals, and solid red lines mark the true parameter values used to generate the data.

3.3 Metropolis Hastings algorithm results

Figure 8 shows the posterior distributions for α, β_1, β_2 and τ obtained from the Metropolis–Hastings sampler. For all four parameters the posterior medians are close to the true values, and the 95% credible intervals are similar in width to those from the Gibbs sampler. This agrees with the numerical summaries in Table 3, where the MH posterior means are very close to the maximum likelihood and Gibbs estimates, and the overall RMSE for MH is only slightly larger than for the two Gibbs implementations.

The main difference between MH and Gibbs is in the mixing behaviour of the chains. Figure 9 shows that the chains for the four parameters fluctuate around the true values, but successive draws are highly correlated, and the trace for τ has long runs away from the true value. This is confirmed by the autocorrelation functions in Figure 10, where autocorrelation decays much more slowly than for the Gibbs sampler, and remains very high for α and especially τ. In terms of runtime, MH is similar to my Gibbs implementation for 10,000 iterations, but because of the stronger autocorrelation it effectively produces fewer independent posterior samples in the same computing time.

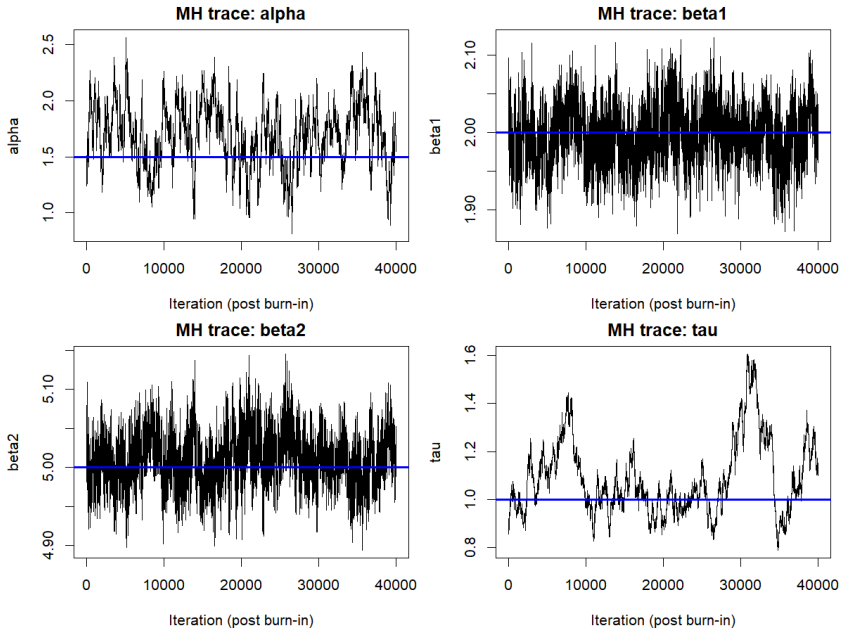


Figure 9: Post–burn-in traceplots for α, β_1, β_2 and τ from the Metropolis–Hastings (MH) sampler. The black lines show the MH draws and the blue horizontal lines indicate the true parameter values. The chains for α, β_1, β_2 fluctuate stably around the true values with no obvious trends, whereas τ shows longer deviations and slower mixing, indicating higher autocorrelation and poorer exploration of its posterior

4 Conclusion

In this project a simple Bayesian linear regression model with two covariates was studied using simulated data with known true parameter values. Several methods for estimating the parameters were compared, including maximum likelihood, a custom Gibbs sampler in R, a custom Metropolis–Hastings (MH) algorithm in R, a custom importance sampling (IS) algorithm, and the `MCMCregress` function from the `MCMCpack` package in R [5].

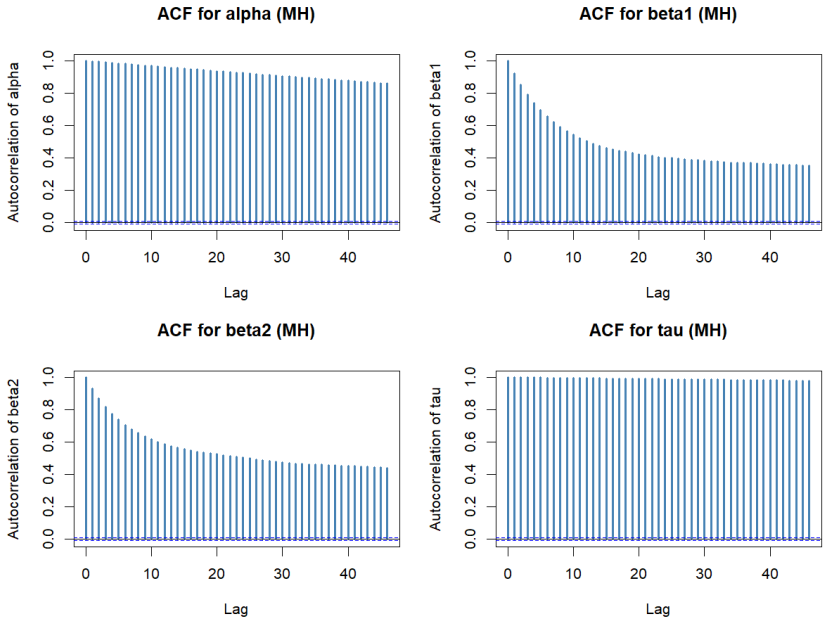


Figure 10: Autocorrelation functions (ACFs) for the post–burn-in samples of α, β_1, β_2, and τ. Autocorrelation decays slowly for β_1 and β_2, and remains very high across all lags for α and τ, indicating strong dependence between successive MH samples and relatively poor mixing compared with the Gibbs sampler

The focus was on posterior accuracy, mixing behaviour of the post–burn-in chains, and computational cost.

For the conjugate model described in Section 2.1, the Gibbs sampler performed best overall among the studied algorithms, as shown in Table 3. It produced posterior distributions that were centred very close to the true parameter values with low variability, and the simulations over 1000 datasets suggested negligible bias in the posterior means. Its post-burn-in chain also had low autocorrelation across lags, indicating good mixing of the Gibbs sampler.

The `MCMCpack` implementation of Gibbs regression gave almost identical results while being about 6.1 times faster for the same number of iterations and burn-in. This is consistent with its design as an efficient, general-purpose MCMC package for Bayesian regression and related models [5].

The Metropolis–Hastings (MH) sampler gave parameter estimates close to the true values, with posterior means and credible intervals similar to those from the Gibbs sampler and maximum likelihood. However, the resulting post-burn-in chain showed stronger autocorrelation, especially for α and τ. This suggests that for a fixed number of iterations, the MH algorithm produced fewer independent posterior samples than our Gibbs sampler, which is consistent with theory for MH algorithms in higher dimensions [6].

In practice, the mixing of the Metropolis–Hastings sampler is also sensitive to the choice of proposal variance. If the proposal is poorly tuned, the chain either accepts too rarely or takes only very small steps, which leads to strong autocorrelation and slow exploration of the posterior as seen in Figures 9 and 10.

In contrast, the importance sampler performed poorly for this model when the prior was used as the proposal. The resulting weights were highly variable, which led to a much

flatter posterior approximation, a biased estimate of τ, and a very long run time of about 936 seconds for 1000 posterior draws based on 80,000 simulated parameter vectors. This illustrates the difficulty of designing effective importance sampling proposals in higher dimensions.

Overall, when conjugate full conditional distributions are available, Gibbs sampling is the most effective choice in this example, while Metropolis–Hastings serves as a more general but less efficient alternative, and importance sampling can perform poorly unless the proposal is very well matched to the target distribution.

5 References

References

[1] *Code Github Repository* https://github.com/ajones39/SOR4008

[2] *Wikipedia - Linear regression* https://en.wikipedia.org/wiki/Linear_regression.

[3] *Wikipedia - Bayesian linear regression* https://en.wikipedia.org/wiki/Bayesian_linear_regression.

[4] *SOR4008 - Bayesian statistics Lecture notes - Hannah Mitchell*

[5] *MCMCpack: Markov Chain Monte Carlo in R - Martin et al (2011)* https://www.jstatsoft.org/article/view/v042i09.

[6] *Gelman et al., Bayesian Data Analysis, 3rd ed., Chapter 11* https://sites.stat.columbia.edu/gelman/book/BDA3.pdf.